

Table of Content

S No	Topic	Page No	Date	Sign
1	To generate and plot basic continuous-time signals using MATLAB	3		
2	To verify the Sampling Theorem and observe the effect of aliasing	6		
3	To compute the N-point DFT of a sequence and plot its magnitude and phase spectrum	9		
4	To perform linear and circular convolution of two sequences	12		
5	To design FIR low-pass and high-pass filters using different window functions	16		
6	To design Butterworth IIR low-pass and high-pass filters for given specifications	19		
7	To compute the Hilbert transform and the analytic signal of a signal	23		

EXPERIMENT 1

Basic Signals

Aim:

To generate and plot basic continuous-time signals (unit impulse, unit step, unit ramp, exponential, rectangular pulse, sinusoidal, square, sawtooth and triangular) using MATLAB.

Apparatus:

- Personal computer
- MATLAB software

Theory:

A continuous-time signal $x(t)$ is defined for every value of time t . A small set of elementary signals forms the building blocks used to describe and analyse more complicated signals and systems. In MATLAB a continuous-time signal is represented on a very fine time grid, so that the plotted samples look like a smooth curve.

Singularity signals

- **Unit impulse:** $\delta(t)$ is zero for $t \neq 0$, is infinitely tall at $t = 0$ and has unit area. It cannot be generated exactly, so it is approximated by a narrow rectangular pulse of width Δ and height $1/\Delta$.
- **Unit step:** $u(t) = 1$ for $t \geq 0$ and 0 for $t < 0$.
- **Unit ramp:** $r(t) = t \cdot u(t)$, i.e. the integral of the unit step.
- **Rectangular pulse:** equal to 1 for $|t| \leq T/2$ and 0 elsewhere.

$$\int_{-\infty}^{\infty} \delta(t) dt = 1, \quad u(t) = \int_{-\infty}^t \delta(\tau) d\tau, \quad \delta(t) = \frac{du(t)}{dt}$$

$$\delta_{\Delta}(t) = \frac{1}{\Delta} \text{rect}\left(\frac{t}{\Delta}\right) \longrightarrow \delta(t) \quad \text{as } \Delta \rightarrow 0$$

Exponential and sinusoidal signals

- **Exponential:** $x(t) = e^{at}u(t)$. It decays for $a < 0$ and grows for $a > 0$ ($a = 0.5$ is used here).
- **Sinusoid:** $x(t) = A \sin(2\pi f_0 t + \phi)$, a periodic signal of frequency f_0 .
- **Square, sawtooth and triangular waves:** the most common periodic test waveforms, each with a period $T_0 = 1/f_0$.

$$x(t + T_0) = x(t) \quad \text{for all } t, \quad f_0 = \frac{1}{T_0}$$

In this program the impulse is approximated by a pulse of width $\Delta = 0.05$ s and height 20, the rectangular pulse has $T = 1$ s, and the periodic waves have $f_0 = 100$ Hz ($T_0 = 10$ ms), with three periods plotted.

MATLAB functions used: plot, subplot, exp, sin, floor, mod, abs.

Procedure:

matlab

```
clc; clear; close all;
t = (-20000:40000)*1e-4;
x = {(t>=0 & t<0.0499)/0.05, double(t>=0), t.*(t>=0), exp(0.5*t).*(t>=0),
double(abs(t)<=0.5)};
s = {'Unit Impulse \delta(t)', 'Unit Step u(t)', 'Unit Ramp r(t)', 'Exponential Signal
e^{0.5t}u(t)', 'Rectangular Pulse'};
figure(1);
for i = 1:5
    subplot(3,2,i); plot(t, x{i}, 'LineWidth', 1.5); title(s{i});
    xlabel('Time (s)'); ylabel('Amplitude'); ylim([-0.1 1.1]*max(x{i})); grid on;
end

tp = (0:2999)*1e-5;
w = 100*tp;
saw = 2*(w - floor(w + 1e-9)) - 1;
y = {sin(2*pi*w), 2*(mod(floor(2*w + 1e-9), 2) == 0) - 1, saw, 2*abs(saw) - 1};
s = {'Sine Wave', 'Square Wave', 'Sawtooth Wave', 'Triangular Wave'};
figure(2);
for i = 1:4
    subplot(2,2,i); plot(tp, y{i}, 'LineWidth', 1.5); title(s{i});
    xlabel('Time (s)'); ylabel('Amplitude'); ylim([-1.2 1.2]); grid on;
end
```

Output:

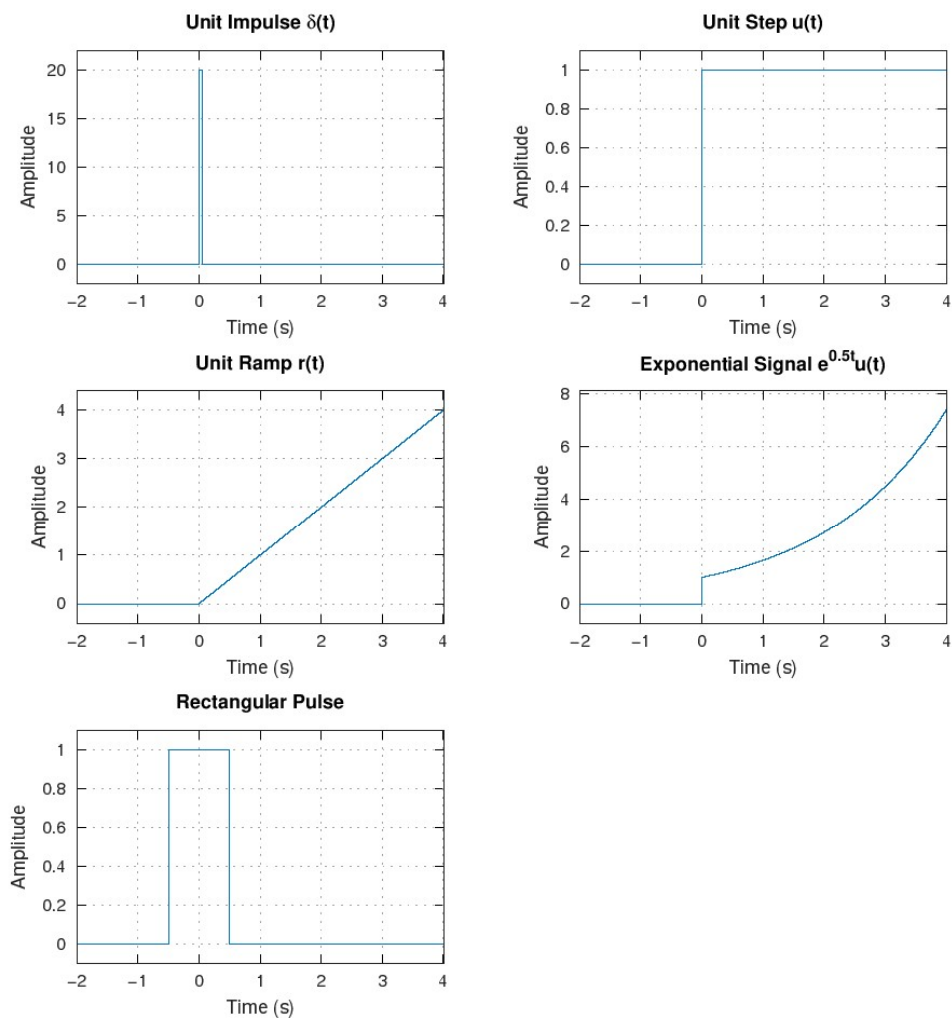


Fig. 1.1: Singularity, exponential and rectangular-pulse signals

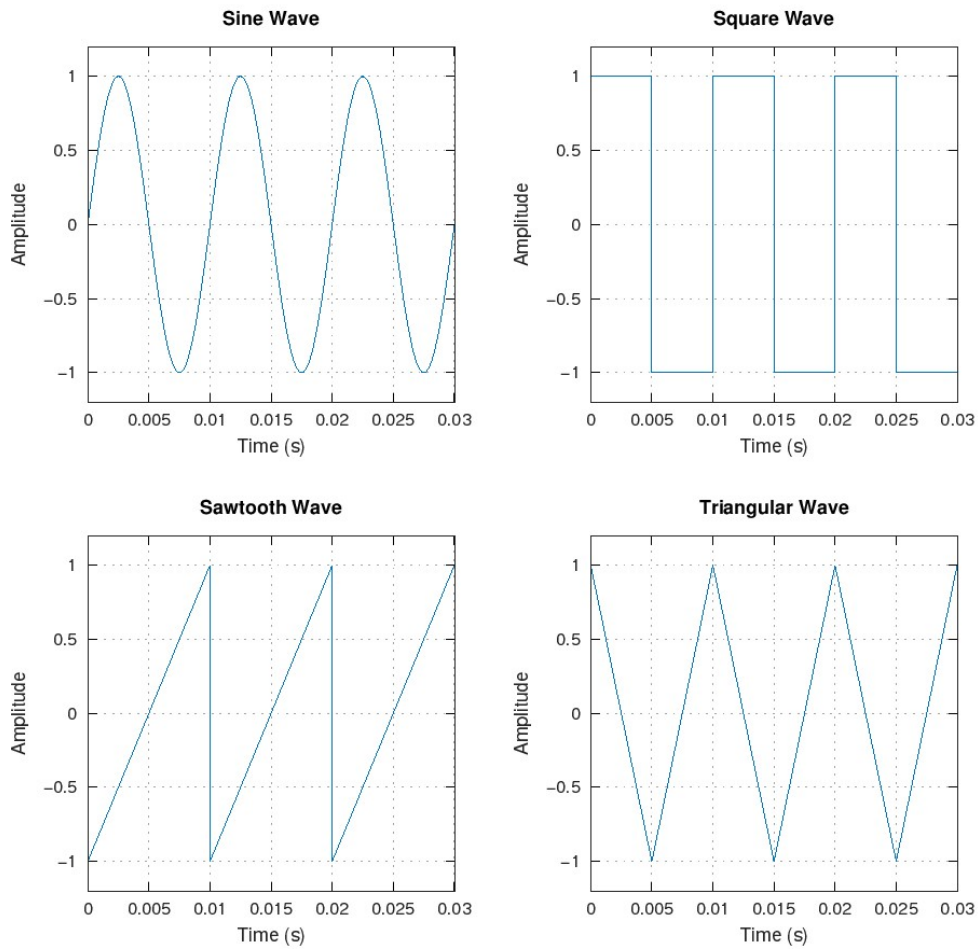


Fig. 1.2: Periodic continuous-time signals

Result:

The basic continuous-time signals (unit impulse, unit step, unit ramp, exponential, rectangular pulse, and sine, square, sawtooth and triangular waves) were generated and plotted using MATLAB.

EXPERIMENT 2

Sampling Theorem

Aim:

To verify the sampling theorem by sampling a signal at rates above, at and below the Nyquist rate, reconstructing it, and observing the effect of aliasing.

Apparatus:

- Personal computer
- MATLAB software
- Signal Processing Toolbox

Theory:

Sampling theorem: a band-limited signal whose highest frequency component is $f_{\max}$ can be completely recovered from its samples if it is sampled at a rate f_s that is at least twice $f_{\max}$.

$$f_s \geq 2f_{\max}$$

The minimum rate $2f_{\max}$ is called the **Nyquist rate**. Sampling is modelled as multiplication of the analog signal by an impulse train of period $T_s = 1/f_s$:

$$x_s(t) = x(t) \sum_{n=-\infty}^{\infty} \delta(t - nT_s)$$

In the frequency domain this makes the spectrum of the signal repeat around every multiple of f_s :

$$X_s(f) = f_s \sum_{k=-\infty}^{\infty} X(f - kf_s)$$

- **Over-sampling ($f_s > 2f_{\max}$):** the spectral replicas are well separated and the signal can be recovered with a low-pass filter.
- **Critical sampling ($f_s = 2f_{\max}$):** the replicas just touch; recovery is possible only in the limit.
- **Under-sampling ($f_s < 2f_{\max}$):** the replicas overlap, causing **aliasing**. A component of frequency f_m then appears at the alias frequency given below, and the original signal cannot be recovered.

$$f_a = \left| f_m - \text{round}\left(\frac{f_m}{f_s}\right) f_s \right|$$

Reconstruction: an ideal low-pass filter of cut-off $f_s/2$ recovers the signal. In the time domain this corresponds to sinc interpolation of the samples:

$$x_r(t) = \sum_{n=-\infty}^{\infty} x(nT_s) \text{sinc}\left(\frac{t - nT_s}{T_s}\right)$$

In this experiment $x(t) = \cos(2\pi \cdot 50t)$ ($f_m = 50$ Hz, Nyquist rate = 100 Hz) is sampled at 500 Hz, 100 Hz and 60 Hz. A cosine is used because a sine sampled exactly at the Nyquist rate would be sampled only at its zero crossings. The mean square error (MSE) between the original and the reconstructed

signal is calculated over the middle part of the record, to avoid the edge effects of the finite sum. For each sampling rate the program also prints the apparent frequency given by the alias formula.

Procedure:

matlab

```
clc; clear; close all;
fm = 50; T = 0.2;
tc = 0:1/20000:T;
xc = cos(2*pi*fm*tc);
fs = [500 100 60];
lab = {'Over-sampling', 'Critical sampling', 'Under-sampling'};
mid = tc >= 0.05 & tc <= 0.15;

subplot(2,2,1); plot(tc, xc, 'LineWidth', 1.5); title('Continuous signal x(t)');
xlabel('Time (s)'); ylabel('Amplitude'); xlim([0 0.08]); grid on;

fprintf('fm = %d Hz, Nyquist rate = %d Hz\n\n', fm, 2*fm);
fprintf('%-8s %-16s %-10s\n', 'fs(Hz)', 'Apparent f (Hz)', 'MSE');
for i = 1:3
    Ts = 1/fs(i); n = 0:floor(T/Ts); xs = cos(2*pi*fm*n*Ts);
    xr = xs * sinc((tc - n*Ts)/Ts);
    fprintf('%-8d %-16d %-10.5f\n', fs(i), abs(fm - round(fm/fs(i))*fs(i)), mean((xc(mid) -
xr(mid)).^2));
    subplot(2,2,i+1); plot(tc, xc, 'b', tc, xr, 'g--', 'LineWidth', 1.2); hold on;
    stem(n*Ts, xs, 'r', 'filled');
    title(sprintf('%s (f_s = %d Hz)', lab{i}, fs(i))); xlabel('Time (s)');
    ylabel('Amplitude');
    xlim([0 0.08]); ylim([-1.5 3]); grid on;
    if i == 1, legend('Original', 'Reconstructed', 'Samples', 'Location', 'north'); end
end
```

Output:

Command Window

MATLAB Command Window

```
>> exp2_sampling_theorem
fm = 50 Hz, Nyquist rate = 100 Hz
```

fs(Hz)	Apparent f (Hz)	MSE
500	50	0.00000
100	50	0.01674
60	10	1.00849

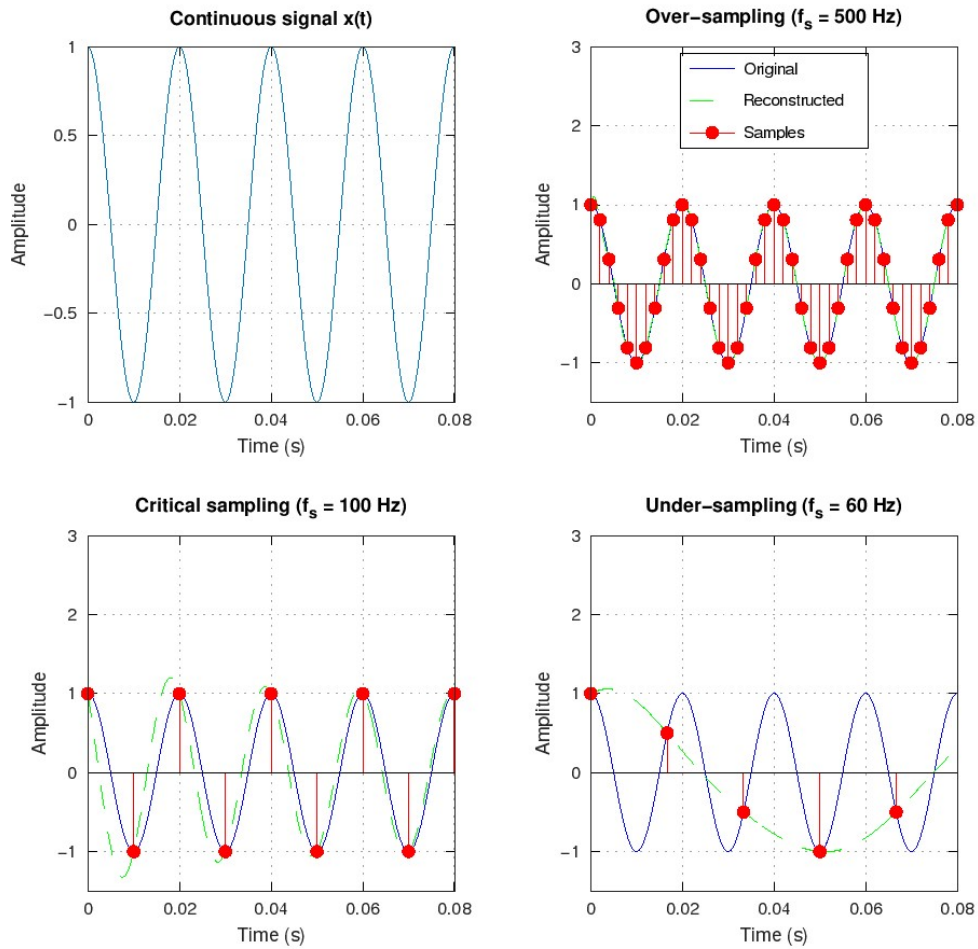


Fig. 2.1: Original, sampled and reconstructed signals for different sampling rates

Result:

The sampling theorem was verified. For $f_s = 500$ Hz and 100 Hz ($f_s \geq 2f_m$) the 50 Hz signal was reconstructed correctly (MSE 0.00000 and 0.01674 respectively), whereas at $f_s = 60$ Hz ($< 2f_m$) aliasing occurred, the MSE rose to 1.00849 and the signal appeared at 10 Hz.

EXPERIMENT 3

N-Point Discrete Fourier Transform

Aim:

To compute the N-point Discrete Fourier Transform (DFT) of a given sequence and to plot its magnitude and phase spectrum.

Apparatus:

- Personal computer
- MATLAB software

Theory:

The Discrete Fourier Transform converts a finite-length sequence $x(n)$ of N samples into N equally spaced samples of its spectrum. The DFT and its inverse (IDFT) are defined as:

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1$$
$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N}, \quad n = 0, 1, \dots, N-1$$

Writing the twiddle factor as $W_N = e^{-j2\pi/N}$, the DFT can be expressed in matrix form, where W_N is an $N \times N$ matrix. This is the method used in the program, so the sequence must contain exactly N samples:

$$W_N = e^{-j2\pi/N} \Rightarrow X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn}$$
$$\mathbf{X} = \mathbf{W}_N \mathbf{x} \quad \text{where} \quad (\mathbf{W}_N)_{kn} = W_N^{kn}$$

$X(k)$ is complex, so the spectrum is described by a magnitude spectrum and a phase spectrum. The index k corresponds to the frequency f_k , where f_s is the sampling frequency (in the program the spectra are plotted against k):

$$|X(k)| = \sqrt{\text{Re}^2[X(k)] + \text{Im}^2[X(k)]}, \quad \angle X(k) = \tan^{-1} \frac{\text{Im}[X(k)]}{\text{Re}[X(k)]}, \quad f_k = \frac{k f_s}{N}$$

Important properties

- **Periodicity:** $X(k + N) = X(k)$ and $x(n + N) = x(n)$.
- **Symmetry:** for a real sequence, $X(N - k) = X^*(k)$; the magnitude is even and the phase is odd about $N/2$.
- **Frequency resolution:** the spacing between DFT bins is $\Delta f = f_s/N$.
- **Computation:** the direct DFT needs N^2 complex multiplications, while the FFT algorithm needs only $(N/2)\log_2 N$.

For $x(n) = \{1, 2, 3, 4, 5, 6, 7, 8\}$ the 8-point DFT is $X(0) = 36$ (the sum of the samples) and $X(k) = -4 + j4\cot(\pi k/8)$ for $k = 1$ to 7. The program also compares its DFT with MATLAB's built-in `fft`.

Procedure:

matlab

```
clc; clear; close all;
N = input('Enter the number of points of the DFT, N = ');
x = input('Enter the sequence x(n) = ');
n = 0:N-1;
X = x * exp(-1j*2*pi/N).^(n'*n);

fprintf('\n%3s %22s %12s %14s\n', 'k', 'X(k)', '|X(k)|', 'Phase (deg)');
for k = n
    fprintf('%3d %11.4f %10.4fj %12.4f %14.4f\n', k, real(X(k+1)), imag(X(k+1)),
    abs(X(k+1)), angle(X(k+1))*180/pi);
end
fprintf('\nMax difference between this DFT and fft = %.2e\n', max(abs(X - fft(x, N))));

subplot(3,1,1); stem(n, x, 'filled'); title('Input sequence x(n)'); xlabel('n');
ylabel('x(n)'); grid on;
subplot(3,1,2); stem(n, abs(X), 'filled'); title('Magnitude spectrum'); xlabel('k');
ylabel('|X(k)|'); grid on;
subplot(3,1,3); stem(n, angle(X)*180/pi, 'filled'); title('Phase spectrum'); xlabel('k');
ylabel('Phase (degrees)'); grid on;
```

Output:

Command Window

MATLAB Command Window

```
>> exp3_dft
Enter the number of points of the DFT, N = 8
Enter the sequence x(n) = [1 2 3 4 5 6 7 8]

    k          X(k)      |X(k)|   Phase (deg)
    0    36.0000   +0.0000j    36.0000     0.0000
    1    -4.0000   +9.6569j    10.4525    112.5000
    2    -4.0000   +4.0000j     5.6569    135.0000
    3    -4.0000   +1.6569j     4.3296    157.5000
    4    -4.0000   -0.0000j     4.0000   -180.0000
    5    -4.0000   -1.6569j     4.3296   -157.5000
    6    -4.0000   -4.0000j     5.6569   -135.0000
    7    -4.0000   -9.6569j    10.4525   -112.5000
```

```
Max difference between this DFT and fft = 6.70e-14
```

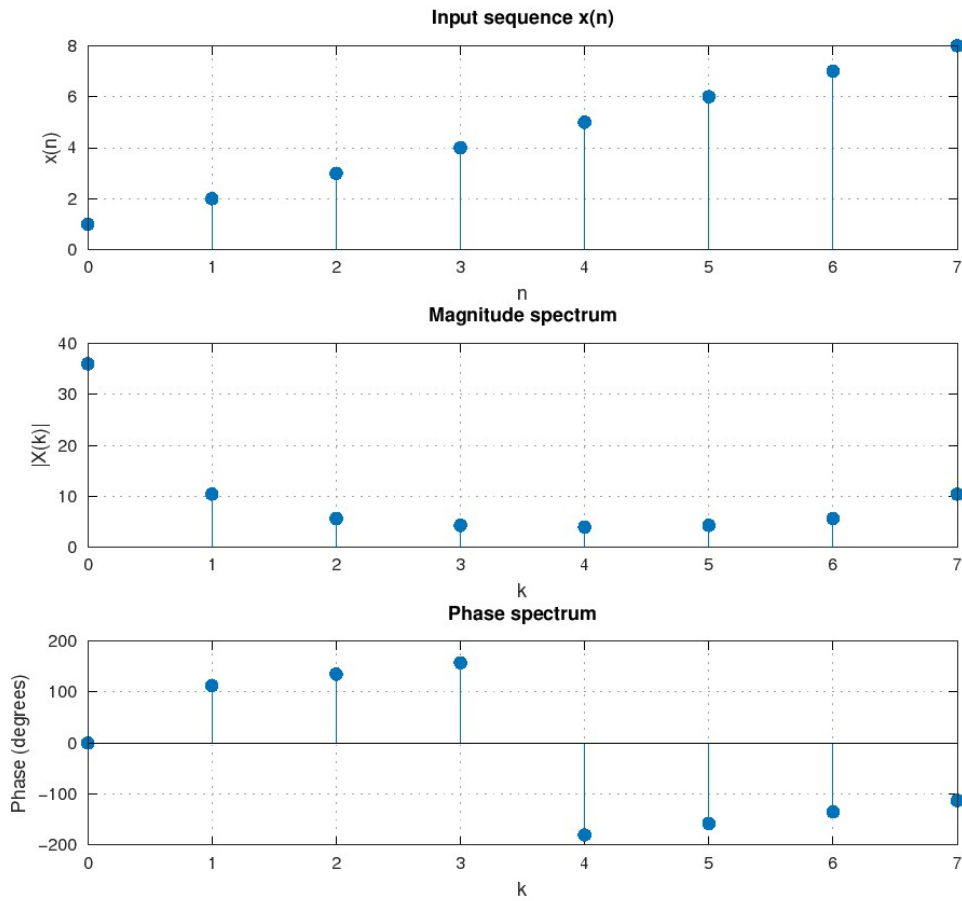


Fig. 3.1: 8-point DFT: input sequence, magnitude spectrum and phase spectrum

Result:

The 8-point DFT of $x(n) = \{1, 2, 3, 4, 5, 6, 7, 8\}$ was computed using the twiddle-factor matrix and its magnitude and phase spectra were plotted: $X(0) = 36$ and $X(1) = -4 + j9.6569$. The results agreed with the built-in fft (difference below 10^{-12}).

EXPERIMENT 4

Linear and Circular Convolution

Aim:

To perform linear convolution and circular convolution of two given sequences and to show that circular convolution equals linear convolution when $N \geq L_x + L_h - 1$.

Apparatus:

- Personal computer
- MATLAB software
- Signal Processing Toolbox

Theory:

Linear convolution gives the output of a linear time-invariant (LTI) system with impulse response $h(n)$ for an input $x(n)$. If $x(n)$ has length L_x and $h(n)$ has length L_h , the output has length $L_x + L_h - 1$.

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k) h(n - k)$$

Circular convolution of two N -point sequences is defined using a circular (modulo- N) shift of one of the sequences. The shorter sequence is zero padded to N points when the lengths are unequal.

$$y_c(n) = \sum_{k=0}^{N-1} x(k) h((n - k) \bmod N), \quad n = 0, 1, \dots, N - 1$$

By the DFT property, circular convolution in the time domain corresponds to multiplication in the frequency domain, so it can also be computed using the DFT and IDFT:

$$Y(k) = X(k) H(k) \quad \Rightarrow \quad y_c(n) = \text{IDFT}\{X(k)H(k)\}$$

When $N < L_x + L_h - 1$, the tail of the linear convolution wraps around and adds to the beginning of the output (time-domain aliasing):

$$y_c(n) = \sum_{m=-\infty}^{\infty} y(n + mN)$$

Hence circular convolution is identical to linear convolution only if $N \geq L_x + L_h - 1$. This is why both sequences are zero padded before using the DFT to implement linear filtering (for example in the overlap-add and overlap-save methods).

For $x(n) = \{4, 5, 6, 7\}$ and $h(n) = \{2, 3, 4\}$ the linear convolution is $\{8, 22, 43, 52, 45, 28\}$. The 4-point circular convolution is $\{53, 50, 43, 52\}$, because the last two samples (45 and 28) wrap around and add to the first two ($8 + 45 = 53$ and $22 + 28 = 50$). In MATLAB, `conv` gives the linear convolution and `cconv(x, h, N)` gives the N -point circular convolution. In the program N is entered by the user ($N = 4$ here), and the circular convolution with $N = L_x + L_h - 1$ is also calculated for comparison.

Procedure:

matlab

```
clc; clear; close all;
x = input('Enter the sequence x(n) = ');
h = input('Enter the sequence h(n) = ');
N = input('Enter the length N of the circular convolution = ');
L = length(x) + length(h) - 1;

yl = conv(x, h);
yc = cconv(x, h, N);
yf = cconv(x, h, L);
fprintf('\n%-34s = [ %s]\n', 'Linear convolution', sprintf('%g ', yl));
fprintf('\n%-34s = [ %s]\n', sprintf('Circular convolution, N = %d', N), sprintf('%g ', yc));
fprintf('\n%-34s = [ %s]\n', sprintf('Circular convolution, N = %d', L), sprintf('%g ', yf));

S = {x, h, yl, yc};
s = {'Sequence x(n)', 'Sequence h(n)', 'Linear convolution', sprintf('Circular convolution, N = %d', N)};
for i = 1:4
    subplot(2,2,i); stem(0:length(S{i})-1, S{i}, 'filled'); title(s{i});
    xlabel('n'); set(gca, 'XTick', 0:length(S{i})-1); grid on;
end
```

Output:

Output 1: $x(n) = \{4, 5, 6, 7\}$, $h(n) = \{2, 3, 4\}$, $N = 4$

MATLAB Command Window

```
>> exp4_convolution
Enter the sequence x(n) = [4 5 6 7]
Enter the sequence h(n) = [2 3 4]
Enter the length N of the circular convolution = 4

Linear convolution          = [ 8 22 43 52 45 28 ]
Circular convolution, N = 4 = [ 53 50 43 52 ]
Circular convolution, N = 6 = [ 8 22 43 52 45 28 ]
```

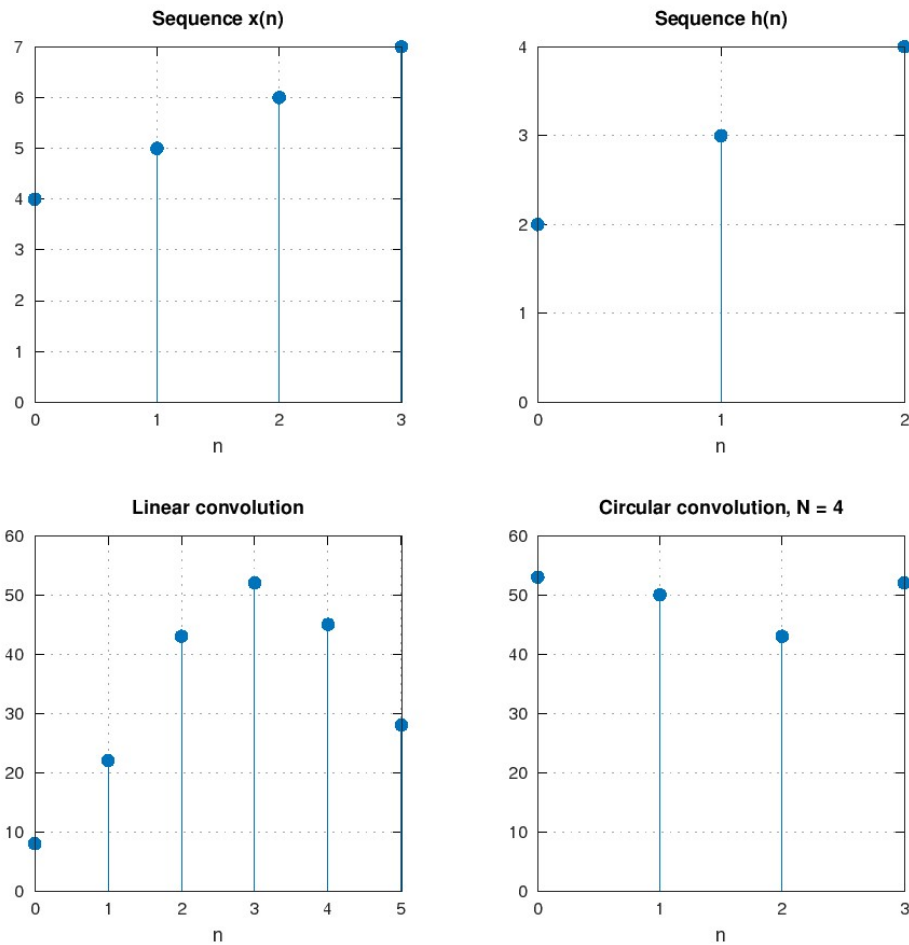


Fig. 4.1: Linear and circular convolution of $x(n) = \{4, 5, 6, 7\}$ and $h(n) = \{2, 3, 4\}$

Output 2: $x(n) = \{4, 3, 2, 1\}$, $h(n) = \{1, 2, 3, 4\}$, $N = 4$

MATLAB Command Window

```
>> exp4_convolution
Enter the sequence x(n) = [4 3 2 1]
Enter the sequence h(n) = [1 2 3 4]
Enter the length N of the circular convolution = 4

Linear convolution           = [ 4 11 20 30 20 11 4 ]
Circular convolution, N = 4  = [ 24 22 24 30 ]
Circular convolution, N = 7  = [ 4 11 20 30 20 11 4 ]
```

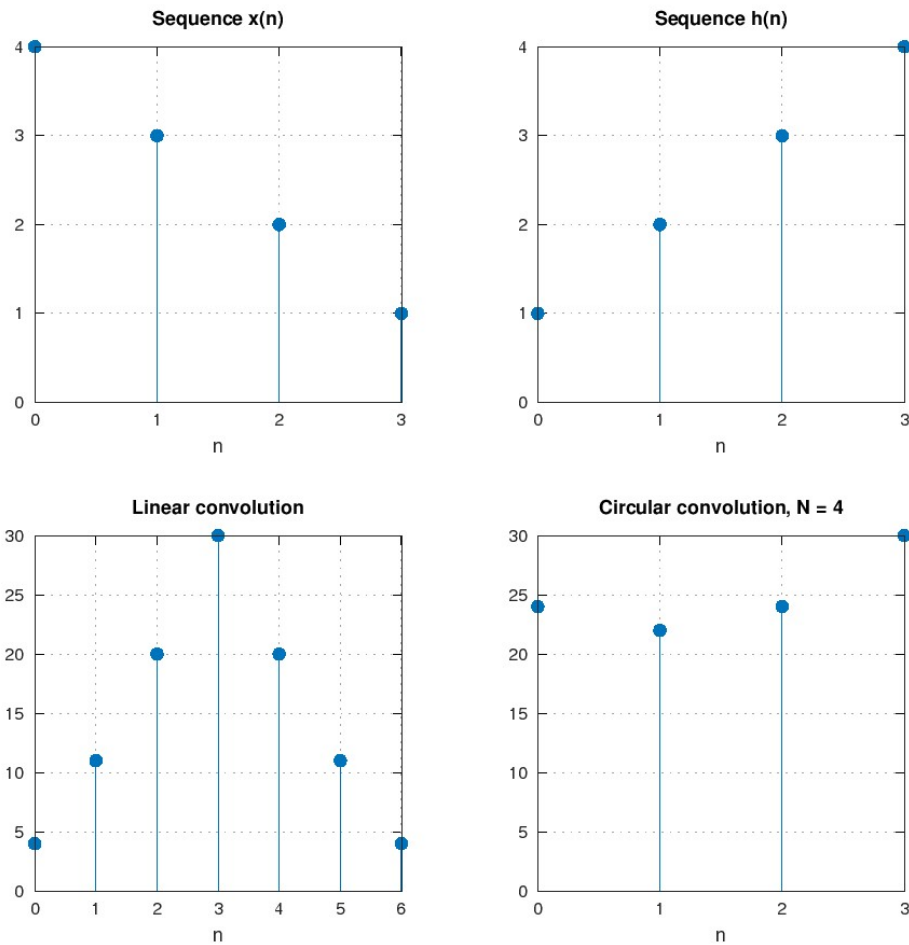


Fig. 4.2: Linear and circular convolution of $x(n) = \{4, 3, 2, 1\}$ and $h(n) = \{1, 2, 3, 4\}$

Result:

Linear and circular convolution of the given sequences were performed. For $x(n) = \{4, 5, 6, 7\}$ and $h(n) = \{2, 3, 4\}$ the linear convolution was $\{8, 22, 43, 52, 45, 28\}$ and the 4-point circular convolution was $\{53, 50, 43, 52\}$; for $x(n) = \{4, 3, 2, 1\}$ and $h(n) = \{1, 2, 3, 4\}$ the 4-point circular convolution was $\{24, 22, 24, 30\}$. Circular convolution with $N = L_x + L_h - 1$ was identical to the linear convolution.

EXPERIMENT 5

FIR Filters

Aim:

To design FIR low-pass and high-pass filters using the window method with rectangular, triangular, Hanning, Hamming and Blackman windows, and to compare their magnitude responses, pass-band ripple and stop-band attenuation.

Apparatus:

- Personal computer
- MATLAB software
- Signal Processing Toolbox

Theory:

An FIR filter has an impulse response $h(n)$ of finite length $M + 1$. Its output depends only on the present and past inputs, and its transfer function has no poles other than at the origin:

$$y(n) = \sum_{k=0}^M h(k) x(n-k) \quad H(z) = \sum_{n=0}^M h(n) z^{-n}$$

- FIR filters are **always stable**.
- They can have **exactly linear phase** when the impulse response is symmetric, giving a constant group delay of $M/2$ samples and no phase distortion.
- They need a higher order than an IIR filter to meet the same magnitude specification.

$$h(n) = h(M-n) \quad \Rightarrow \quad \theta(\omega) = -\omega \frac{M}{2}$$

Window method of design

The impulse response of the ideal filter $h_d(n)$ is infinitely long and non-causal. It is truncated to $M + 1$ samples and delayed by $M/2$ to make it causal. For an ideal low-pass filter with cut-off ω_c :

$$h_d(n) = \frac{\sin[\omega_c(n-\alpha)]}{\pi(n-\alpha)}, \quad \alpha = \frac{M}{2}$$

Abrupt truncation (rectangular window) causes the Gibbs phenomenon: ripple in the pass-band and poor stop-band attenuation. A smoother window $w(n)$ is therefore multiplied with $h_d(n)$:

$$h(n) = h_d(n) w(n), \quad 0 \leq n \leq M$$

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{M}\right) \quad (\text{Hamming})$$

Window	Main-lobe width	Min. stop-band attenuation	Transition width (approx.)
Rectangular	$4\pi/M$	21 dB	$1.8\pi/M$
Triangular	$8\pi/M$	25 dB	$6.1\pi/M$
Hanning	$8\pi/M$	44 dB	$6.2\pi/M$

Window	Main-lobe width	Min. stop-band attenuation	Transition width (approx.)
Hamming	$8\pi/M$	53 dB	$6.6\pi/M$
Blackman	$12\pi/M$	74 dB	$11\pi/M$

A wider main lobe gives a wider transition band, whereas lower side-lobes give better stop-band attenuation. The order M must therefore be large enough for the chosen window to fit the required transition band. The ideal high-pass filter is obtained by subtracting the low-pass response from an all-pass response, which requires an even order M (a symmetric filter with an integer delay).

The MATLAB function `fir1(M, Wn, type, window)` designs the filter, where the cut-off Wn is normalised to the Nyquist frequency $f_s/2$. In this experiment $f_s = 1000$ Hz and $M = 60$, and the low-pass filter has a pass-band edge $f_p = 200$ Hz and a stop-band edge $f_{st} = 300$ Hz. The cut-off is placed at the centre of the transition band, $f_c = (f_p + f_{st})/2 = 250$ Hz, and is used for both the low-pass and the high-pass filter. The transition width is 100 Hz (0.2π rad), which even the Blackman window ($11\pi/M$) can accommodate. For each window the program calculates the pass-band ripple (below f_p) and the stop-band attenuation (above f_{st}) of the low-pass filter.

Procedure:

matlab

```

clc; clear; close all;
fs = 1000; fp = 200; fst = 300; M = 60;
Wn = (fp + fst)/fs;
names = {'Rectangular', 'Triangular', 'Hanning', 'Hamming', 'Blackman'};
w = [rectwin(M+1) triang(M+1) hann(M+1) hamming(M+1) blackman(M+1)];

for i = 1:5
    [HL(:,i), f] = freqz(fir1(M, Wn, 'low', w(:,i)), 1, 512, fs);
    HH(:,i) = freqz(fir1(M, Wn, 'high', w(:,i)), 1, 512, fs);
end
GL = 20*log10(abs(HL) + eps);
GH = 20*log10(abs(HH) + eps);
ripple = max(abs(GL(f <= fp, :)));
atten = -max(GL(f >= fst, :));

fprintf('%-13s %-22s %-22s\n', 'Window', 'Pass-band ripple (dB)', 'Stop-band atten. (dB)');
for i = 1:5
    fprintf('%-13s %-22.3f %-22.2f\n', names{i}, ripple(i), atten(i));
end

subplot(2,1,1); plot(f, GL, 'LineWidth', 1.2); title('FIR low-pass filter');
xlabel('Frequency (Hz)'); ylabel('Gain (dB)'); axis([0 500 -120 5]); grid on;
legend(names, 'Location', 'southwest');
subplot(2,1,2); plot(f, GH, 'LineWidth', 1.2); title('FIR high-pass filter');
xlabel('Frequency (Hz)'); ylabel('Gain (dB)'); axis([0 500 -120 5]); grid on;

```

Output:

Command Window

MATLAB Command Window

```

>> exp5_fir_filters
Window      Pass-band ripple (dB)  Stop-band atten. (dB)
Rectangular  0.335                  30.05
Triangular  0.199                  30.02
Hanning      0.016                  54.87
Hamming      0.020                  57.99

```

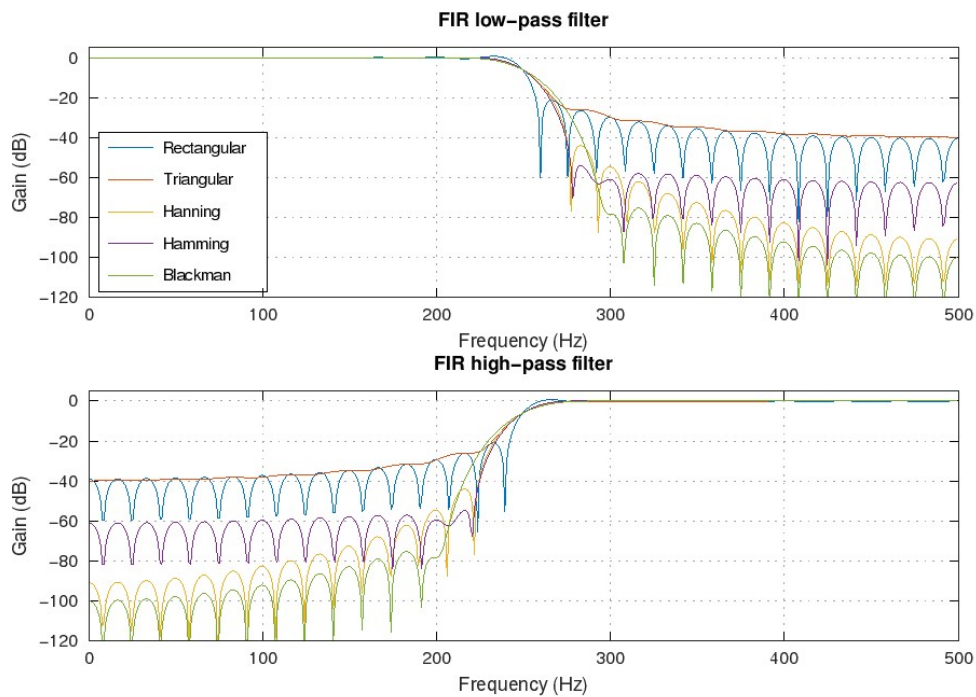


Fig. 5.1: Magnitude responses of the FIR low-pass and high-pass filters designed with different windows

Result:

FIR low-pass and high-pass filters were designed using five different windows. The stop-band attenuation obtained was 30.05 dB (rectangular), 30.02 dB (triangular), 54.87 dB (Hanning), 57.99 dB (Hamming) and 75.26 dB (Blackman), and the pass-band ripple fell from 0.335 dB (rectangular) to 0.002 dB (Blackman).

EXPERIMENT 6

IIR Filters

Aim:

To design Butterworth IIR low-pass and high-pass filters for given specifications and to filter a given sequence with them.

Apparatus:

- Personal computer
- MATLAB software
- Signal Processing Toolbox

Theory:

An IIR filter has an impulse response of infinite duration and is realised recursively, i.e. the output depends on past outputs as well as inputs:

$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}}$$

IIR filters meet a given magnitude specification with a much lower order than FIR filters, but their phase is non-linear and they can be unstable. A digital IIR filter is stable only if all poles lie **inside the unit circle**.

Design procedure

An analog prototype is designed first and then converted to a digital filter using the **bilinear transformation**. It maps the entire $j\Omega$ axis onto one turn of the unit circle, which avoids aliasing but warps the frequency axis, so the edge frequencies are pre-warped before the analog filter is designed. MATLAB does this internally in buttord and butter.

$$s = \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}} \quad \Omega = \frac{2}{T} \tan\left(\frac{\omega}{2}\right)$$

Butterworth filter

The magnitude response of a Butterworth filter is maximally flat in the pass-band and decreases monotonically, without ripple, in the stop-band. Its gain is $1/\sqrt{2}$ (–3 dB) at the cut-off Ω_c and falls by 20N dB per decade, where N is the order:

$$|H(j\Omega)|^2 = \frac{1}{1 + (\Omega/\Omega_c)^{2N}}$$

The minimum order depends on the pass-band edge (Ω_p , ripple R_p) and the stop-band edge (Ω_s , attenuation R_s). The cut-off Ω_c is then placed between the limits set by the pass-band and stop-band conditions; the value that meets the pass-band condition exactly is:

$$N \geq \frac{\log[(10^{0.1R_s} - 1)/(10^{0.1R_p} - 1)]}{2\log(\Omega_s/\Omega_p)}$$

$$\Omega_c = \frac{\Omega_p}{(10^{0.1R_p} - 1)^{1/2N}}$$

The analog poles lie on a semicircle of radius Ω_c in the left half of the s-plane. A high-pass filter is obtained from the low-pass prototype by the low-pass to high-pass transformation, which is done by `butter(N, Wn, 'high')`. For a low-pass filter $f_p < f_{st}$, whereas for a high-pass filter $f_p > f_{st}$.

MATLAB functions used: `buttord`, `butter`, `freqz`, `filter`. The program first asks for the type of filter (1 = low-pass, 2 = high-pass) and then for R_p , R_s , f_p , f_{st} and f_s . It designs the filter and filters a test sequence made of two tones at $f_s/16$ and $0.45f_s$ (500 Hz and 3600 Hz for $f_s = 8000$ Hz). The runs shown use $R_p = 1$ dB, $R_s = 40$ dB and $f_s = 8000$ Hz, with $f_p = 1000$ Hz and $f_{st} = 2000$ Hz for the low-pass filter, and $f_p = 2000$ Hz and $f_{st} = 1000$ Hz for the high-pass filter.

Procedure:

matlab

```
clc; clear; close all;
c = input('Enter choice of filter 1. LPF 2. HPF : ');
Rp = input('Enter the pass-band ripple Rp (dB) = ');
Rs = input('Enter the stop-band attenuation Rs (dB) = ');
fp = input('Enter the pass-band edge frequency (Hz) = ');
fst = input('Enter the stop-band edge frequency (Hz) = ');
fs = input('Enter the sampling frequency (Hz) = ');

[N, Wn] = buttord(fp/(fs/2), fst/(fs/2), Rp, Rs);
type = {'low', 'high'};
[b, a] = butter(N, Wn, type{c});
[H, f] = freqz(b, a, 1024, fs);
G = 20*log10(abs(freqz(b, a, [fp fst], fs)));

fprintf('\nOrder N = %d, cut-off = %.1f Hz\n', N, Wn*fs/2);
fprintf('Gain at %d Hz = %.2f dB\n', fp, G(1));
fprintf('Gain at %d Hz = %.2f dB\n', fst, G(2));

t = 0:1/fs:0.05;
x = sin(2*pi*(fs/16)*t) + sin(2*pi*(0.45*fs)*t);
y = filter(b, a, x);

subplot(3,1,1); plot(f, 20*log10(abs(H) + eps), 'LineWidth', 1.5); title('Magnitude response');
xlabel('Frequency (Hz)'); ylabel('Gain (dB)'); axis([0 fs/2 -100 5]); grid on;
subplot(3,1,2); plot(t, x, 'LineWidth', 1.2); title('Input sequence');
xlabel('Time (s)'); ylabel('Amplitude'); grid on;
subplot(3,1,3); plot(t, y, 'LineWidth', 1.2); title('Output sequence');
xlabel('Time (s)'); ylabel('Amplitude'); grid on;
```

Output:

Output 1: Butterworth low-pass filter

MATLAB Command Window

```
>> exp6_iir_filters
Enter choice of filter 1. LPF 2. HPF : 1
Enter the pass-band ripple Rp (dB) = 1
Enter the stop-band attenuation Rs (dB) = 40
Enter the pass-band edge frequency (Hz) = 1000
Enter the stop-band edge frequency (Hz) = 2000
Enter the sampling frequency (Hz) = 8000

Order N = 6, cut-off = 1105.4 Hz
Gain at 1000 Hz = -1.00 dB
Gain at 2000 Hz = -40.07 dB
```

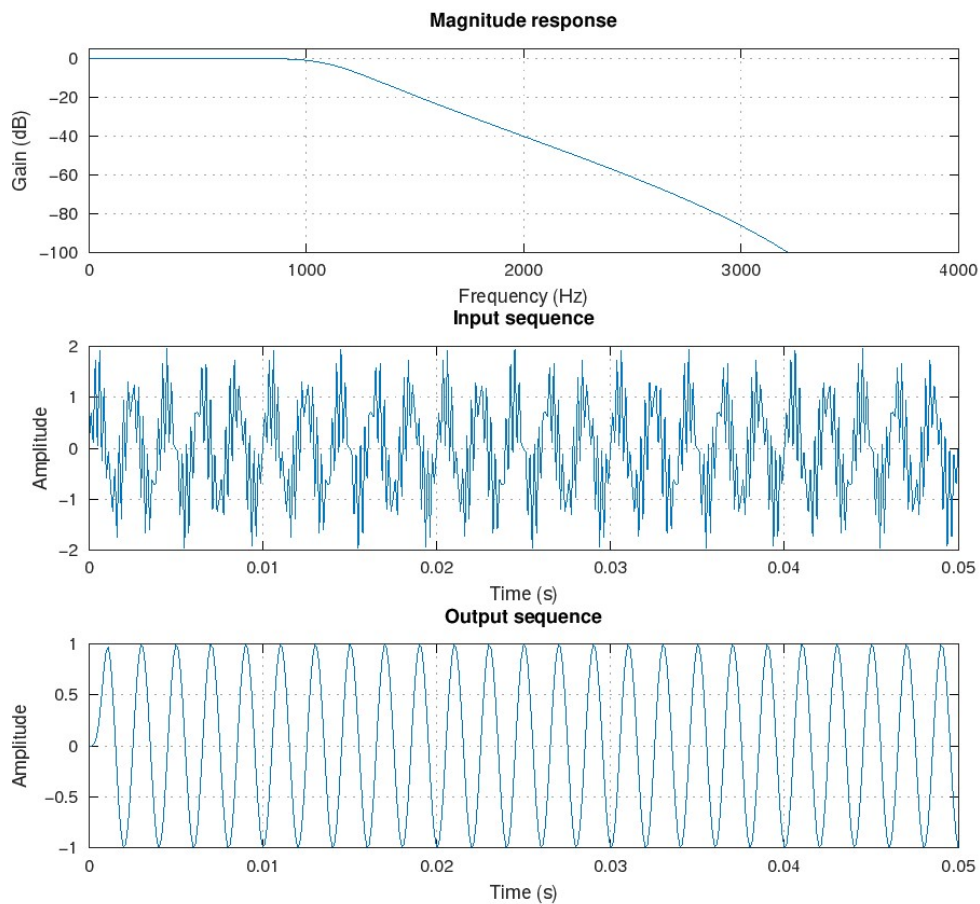


Fig. 6.1: Magnitude response of the Butterworth low-pass filter and filtering of the test sequence

Output 2: Butterworth high-pass filter

MATLAB Command Window

```
>> exp6_iir_filters
Enter choice of filter 1. LPF 2. HPF : 2
Enter the pass-band ripple Rp (dB) = 1
Enter the stop-band attenuation Rs (dB) = 40
Enter the pass-band edge frequency (Hz) = 2000
Enter the stop-band edge frequency (Hz) = 1000
Enter the sampling frequency (Hz) = 8000
```

```
Order N = 6, cut-off = 1856.9 Hz
Gain at 2000 Hz = -1.00 dB
Gain at 1000 Hz = -40.07 dB
```

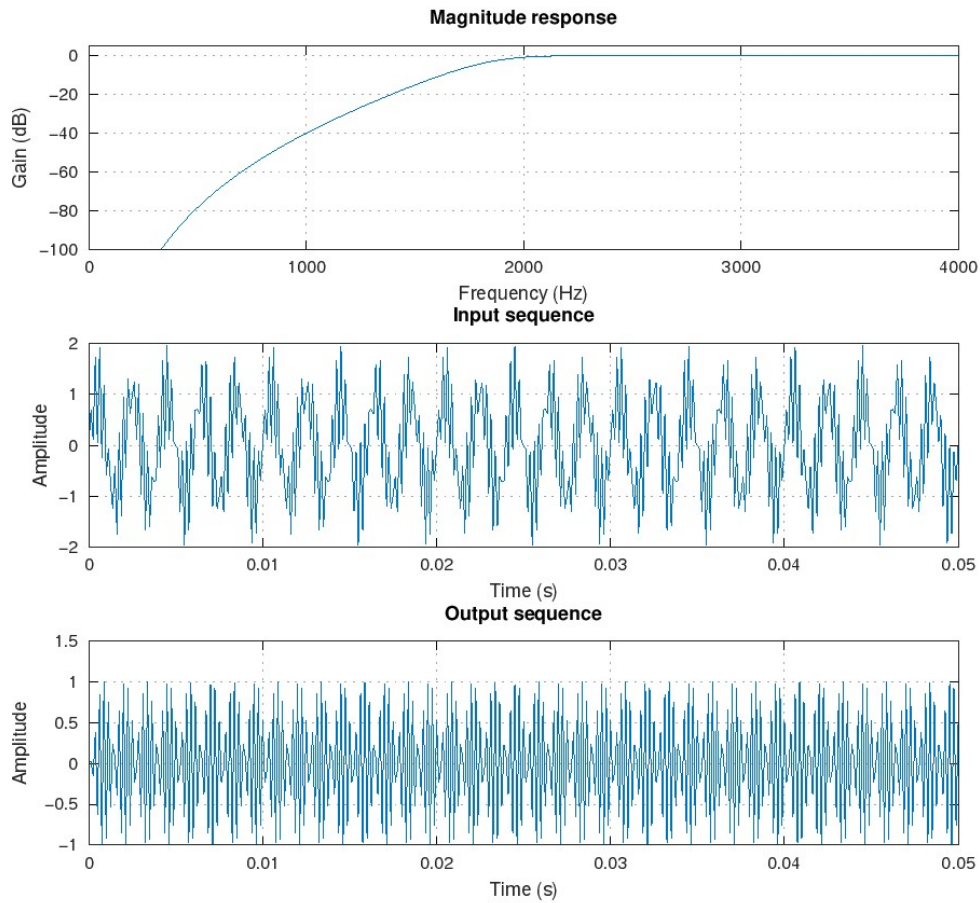


Fig. 6.2: Magnitude response of the Butterworth high-pass filter and filtering of the test sequence

Result:

A sixth-order Butterworth low-pass filter and a sixth-order Butterworth high-pass filter were designed for $R_p = 1$ dB and $R_s = 40$ dB at $f_s = 8000$ Hz. Both met the specification: the gain at the pass-band edge was not below -1 dB and the gain at the stop-band edge was not above -40 dB. The low-pass filter removed the 3600 Hz tone from the test sequence, while the high-pass filter removed the 500 Hz tone.

EXPERIMENT 7

Hilbert Transform

Aim:

To compute the Hilbert transform and the analytic signal of a signal using MATLAB.

Apparatus:

- Personal computer
- MATLAB software
- Signal Processing Toolbox

Theory:

The Hilbert transform $\hat{x}(t)$ of a signal $x(t)$ is obtained by convolving $x(t)$ with $1/(\pi t)$. It is a linear operation that shifts the phase of every frequency component by 90° without changing its magnitude.

$$\hat{x}(t) = \frac{1}{\pi} \text{P. V.} \int_{-\infty}^{\infty} \frac{x(\tau)}{t-\tau} d\tau = x(t) * \frac{1}{\pi t}$$

It can be viewed as passing $x(t)$ through a filter (Hilbert transformer) with frequency response:

$$H(j\omega) = -j \operatorname{sgn}(\omega) \quad \Rightarrow \quad |H| = 1, \quad \angle H = \mp 90^\circ$$

Hence positive frequencies are shifted by -90° and negative frequencies by $+90^\circ$. For example:

$$\mathcal{H}\{\cos \omega_0 t\} = \sin \omega_0 t \quad \mathcal{H}\{\sin \omega_0 t\} = -\cos \omega_0 t$$

Analytic signal

Combining a real signal with its Hilbert transform gives the analytic signal, whose spectrum is zero for negative frequencies (the spectrum is doubled for $f > 0$). The MATLAB function `hilbert` returns this complex analytic signal; the Hilbert transform itself is its imaginary part.

$$x_a(t) = x(t) + j\hat{x}(t)$$

Discrete-time implementation

The ideal discrete-time Hilbert transformer has the impulse response given below. In the DFT domain it multiplies the positive-frequency bins by $-j$ and the negative-frequency bins by $+j$; the resulting phase of the Hilbert transformer is plotted.

$$h(n) = \frac{2}{\pi n} \sin^2\left(\frac{\pi n}{2}\right) \quad (n \neq 0), \quad h(0) = 0$$

In this experiment $x(t) = \cos(2\pi \cdot 50t)$ is sampled at $f_s = 1000$ Hz for 1 s ($N = 1000$ samples). The function `hilbert` returns the analytic signal x_a , and its imaginary part, the Hilbert transform, is compared with $\sin(2\pi \cdot 50t)$. The Hilbert transformer response H is built with zero at DC and at the Nyquist bin, $-j$ for the positive-frequency bins and $+j$ for the negative-frequency bins, and its phase is plotted against frequency.

Applications: single-sideband (SSB) modulation and quadrature signal generation.

Procedure:

matlab

```
clc; clear; close all;
fs = 1000; t = 0:1/fs:1-1/fs; N = length(t);
x = cos(2*pi*50*t);
xh = imag(hilbert(x));
fprintf('Max error between hilbert(x) and sin(2*pi*50*t) = %.2e\n', max(abs(xh -
sin(2*pi*50*t))));

H = zeros(1, N);
H(2:N/2) = -1j;
H(N/2+2:N) = 1j;

subplot(2,1,1); plot(t(1:100), x(1:100), 'b', t(1:100), xh(1:100), 'r', 'LineWidth', 1.5);
title('x(t) and its Hilbert transform'); xlabel('Time (s)'); ylabel('Amplitude');
legend('x(t) = cos', 'H[x(t)] = sin'); grid on;
subplot(2,1,2); plot((-N/2:N/2-1)*fs/N, fftshift(angle(H))*180/pi, 'LineWidth', 1.5);
title('Phase of Hilbert transformer H(f)'); xlabel('Frequency (Hz)'); ylabel('Phase (degrees)');
ylim([-120 120]); grid on;
```

Output:

Command Window

MATLAB Command Window

```
>> exp7_hilbert_transform
Max error between hilbert(x) and sin(2*pi*50*t) = 4.82e-14
```

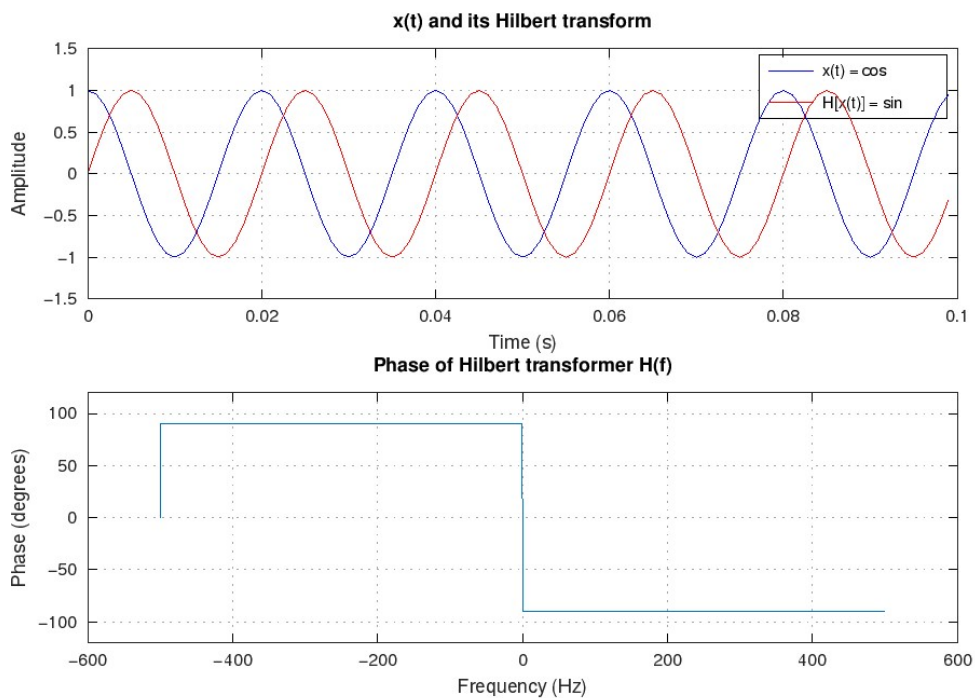


Fig. 7.1: Hilbert transform of $\cos(2\pi \cdot 50t)$ and phase of the Hilbert transformer

Result:

The Hilbert transform of $\cos(2\pi \cdot 50t)$ was obtained as $\sin(2\pi \cdot 50t)$ using the built-in hilbert function (error of the order of 10^{-14}), and the Hilbert transformer showed a -90° phase shift for positive frequencies and $+90^\circ$ for negative frequencies.